06 — Array Types in PostgreSQL

Table of Contents

1. Learning Objectives
2. Array Type Overview
3. Declaring Arrays
4. Array Literals and Input Syntax
5. Accessing Array Elements
6. Array Operators
7. Array Functions
8. Multi-Dimensional Arrays
9. Indexing Arrays
10. Arrays vs JSONB vs Normalized Tables
11. SQL Examples
12. Common Mistakes
13. Best Practices
14. Performance Considerations
15. Interview Questions & Answers
16. Exercises with Solutions
17. Cross-References

Learning Objectives

After completing this section you will be able to:

- Declare and populate arrays of any PostgreSQL type
- Query, modify, and aggregate array data using operators and functions
- Create GIN indexes for efficient array containment queries
- Know when to use arrays vs normalized junction tables vs JSONB
- Avoid common pitfalls of array-based design

Array Type Overview

In PostgreSQL, **any data type can be made into an array** by appending `[]` to the type name. Arrays are:

- **Typed:** every element must be the same type
- **1-indexed by default** (not 0-indexed like most languages)
- **Multi-dimensional:** `INTEGER[][]` is valid
- **Dynamic:** can grow and shrink without pre-declaring size

```
Array Internals:
┌──────────────────────────────────────────────────────┐
│ Array header: dimensions, lower bounds, element type OID  │
├──────────────────────────────────────────────────────┤
│ Element 1 │ Element 2 │ Element 3 │ NULL bits ...         │
└──────────────────────────────────────────────────────┘
```

Arrays are stored inline in the row for small arrays; TOAST-ed for large ones.

Declaring Arrays

```
-- Append [] to any type
col_name  INTEGER[]
col_name  TEXT[]
col_name  NUMERIC(10,4)[]
col_name  TIMESTAMPTZ[]
col_name  UUID[]

-- Specifying dimensions (ignored by PostgreSQL, any dimension allowed)
col_name  INTEGER[3]         -- same as INTEGER[] internally
col_name  INTEGER[3][4]      -- same as INTEGER[][] internally

-- SQL standard ARRAY syntax
col_name  INTEGER ARRAY
col_name  INTEGER ARRAY[10]  -- still no enforcement
```

Array Literals and Input Syntax

```
-- Curly brace notation (PostgreSQL native)
SELECT ARRAY[1, 2, 3];
SELECT '{1, 2, 3}'::INTEGER[];
SELECT ARRAY['a', 'b', 'c'];
SELECT '{"a","b","c"}'::TEXT[];

-- Empty array
SELECT ARRAY[]::INTEGER[];
SELECT '{}'::TEXT[];

-- Array with NULLs
SELECT ARRAY[1, NULL, 3];
SELECT '{1,NULL,3}'::INTEGER[];

-- From query results
SELECT ARRAY(SELECT user_id FROM users WHERE is_active);

-- Multi-dimensional
SELECT ARRAY[[1,2],[3,4]];  -- 2D: {{1,2},{3,4}}

-- Nested arrays in literals
SELECT '{{1,2},{3,4}}'::INTEGER[][];
```

Accessing Array Elements

PostgreSQL arrays are 1-indexed (element 1 is the first element).

```sql
-- Single element access
SELECT ARRAY[10, 20, 30][1];   -- 10
SELECT ARRAY[10, 20, 30][2];   -- 20
SELECT ARRAY[10, 20, 30][3];   -- 30
SELECT ARRAY[10, 20, 30][4];   -- NULL (out of bounds, no error!)
SELECT ARRAY[10, 20, 30][0];   -- NULL (no element 0 by default)

-- Slice: [start:end] (inclusive)
SELECT ARRAY[10, 20, 30, 40, 50][2:4];  -- {20,30,40}
SELECT ARRAY[10, 20, 30, 40, 50][:3];   -- {10,20,30}
SELECT ARRAY[10, 20, 30, 40, 50][3:];   -- {30,40,50}

-- From table column
SELECT tags[1] AS first_tag FROM products;
SELECT tags[1:3] AS first_three_tags FROM products;

-- 2D array access
SELECT ('{{1,2},{3,4}}'::INTEGER[][])[1][2];  -- 2
SELECT ('{{1,2},{3,4}}'::INTEGER[][])[2][1];  -- 3
```

Array Operators

```sql
-- Concatenation
SELECT ARRAY[1,2] || ARRAY[3,4];        -- {1,2,3,4}
SELECT ARRAY[1,2] || 3;                  -- {1,2,3}  (append element)
SELECT 0 || ARRAY[1,2];                  -- {0,1,2}  (prepend element)

-- Equality
SELECT ARRAY[1,2,3] = ARRAY[1,2,3];     -- TRUE
SELECT ARRAY[1,2,3] = ARRAY[1,3,2];     -- FALSE (order matters)

-- Comparison (lexicographic)
SELECT ARRAY[1,2,3] < ARRAY[1,2,4];     -- TRUE
SELECT ARRAY[1,2] < ARRAY[1,2,3];       -- TRUE (shorter is less)

-- Containment
SELECT ARRAY[1,2,3] @> ARRAY[1,2];      -- TRUE (left contains right)
SELECT ARRAY[1,2] <@ ARRAY[1,2,3];      -- TRUE (left is contained in right)

-- Overlap (any element in common)
SELECT ARRAY[1,2,3] && ARRAY[2,4,6];    -- TRUE (2 is in common)
SELECT ARRAY[1,3,5] && ARRAY[2,4,6];    -- FALSE (no overlap)

-- ANY / ALL
SELECT 3 = ANY(ARRAY[1,2,3,4]);         -- TRUE
SELECT 5 = ANY(ARRAY[1,2,3,4]);         -- FALSE
SELECT 3 > ALL(ARRAY[1,2,3]);           -- FALSE (3 is not > 3)
SELECT 4 > ALL(ARRAY[1,2,3]);           -- TRUE
```

```sql
-- IN using ANY (more idiomatic for arrays)
SELECT * FROM products WHERE 'red' = ANY(tags);

-- Unnest: expand array to rows
SELECT unnest(ARRAY[1,2,3]);  -- rows: 1, 2, 3
```

Array Functions

```sql
-- Length
SELECT array_length(ARRAY[1,2,3], 1);      -- 3 (dimension 1)
SELECT array_length(ARRAY[[1,2],[3,4]], 1); -- 2 (rows in 2D)
SELECT array_length(ARRAY[[1,2],[3,4]], 2); -- 2 (cols in 2D)
SELECT cardinality(ARRAY[1,2,3]);           -- 3 (total elements, any dimension)

-- Searching
SELECT array_position(ARRAY['a','b','c'], 'b');     -- 2
SELECT array_positions(ARRAY[1,2,1,3,1], 1);        -- {1,3,5}

-- Modification
SELECT array_append(ARRAY[1,2], 3);        -- {1,2,3}
SELECT array_prepend(0, ARRAY[1,2]);       -- {0,1,2}
SELECT array_remove(ARRAY[1,2,3,2,1], 2); -- {1,3,1}
SELECT array_replace(ARRAY[1,2,3], 2, 99);-- {1,99,3}
SELECT array_cat(ARRAY[1,2], ARRAY[3,4]); -- {1,2,3,4}

-- Slicing and dimension info
SELECT array_dims(ARRAY[1,2,3]);            -- '[1:3]'
SELECT array_lower(ARRAY[1,2,3], 1);       -- 1
SELECT array_upper(ARRAY[1,2,3], 1);       -- 3
SELECT array_ndims(ARRAY[[1,2],[3,4]]);    -- 2

-- Conversion
SELECT array_to_string(ARRAY[1,2,3], ',');          -- '1,2,3'
SELECT array_to_string(ARRAY[1,NULL,3], ',', 'X');  -- '1,X,3'
SELECT string_to_array('a,b,c', ',');               -- {a,b,c}
SELECT string_to_array('a,,c', ',', '');            -- {a,NULL,c}

-- Fill array
SELECT array_fill(0, ARRAY[3]);            -- {0,0,0}
SELECT array_fill(0, ARRAY[3,4]);          -- 3x4 2D array of zeros

-- Aggregation
SELECT array_agg(user_id ORDER BY created_at) FROM users; -- array of user IDs
SELECT array_agg(DISTINCT country_code) FROM users;        -- unique country codes
```

unnest with ordinality

```sql
-- Expand array with position
SELECT ordinality, val
```

```
FROM unnest(ARRAY['a','b','c']) WITH ORDINALITY AS t(val, ordinality);
-- 1 | a
-- 2 | b
-- 3 | c

-- Expand multiple arrays in parallel (PostgreSQL 9.4+)
SELECT a, b
FROM unnest(ARRAY[1,2,3], ARRAY['x','y','z']) AS t(a, b);
-- 1 | x
-- 2 | y
-- 3 | z
```

Multi-Dimensional Arrays

```
-- 2D array example: matrix
CREATE TABLE matrices (
    matrix_id   SERIAL PRIMARY KEY,
    name        TEXT NOT NULL,
    data        DOUBLE PRECISION[][]
);

INSERT INTO matrices (name, data) VALUES
('identity_3x3', '{{1,0,0},{0,1,0},{0,0,1}}'::DOUBLE PRECISION[][]);

-- Access element (row 2, col 3)
SELECT data[2][3] FROM matrices WHERE name = 'identity_3x3';  -- 0

-- Unnest all elements
SELECT i, j, data[i][j] AS value
FROM matrices,
     generate_series(1, array_length(data, 1)) AS i,
     generate_series(1, array_length(data, 2)) AS j;
```

Indexing Arrays

GIN index for array containment and overlap

```
-- Full GIN index: supports @>, <@, &&, = ANY
CREATE INDEX idx_products_tags ON products USING GIN (tags);

-- Query that uses GIN index
EXPLAIN SELECT * FROM products WHERE tags @> ARRAY['sale'];
EXPLAIN SELECT * FROM products WHERE 'featured' = ANY(tags);
EXPLAIN SELECT * FROM products WHERE tags && ARRAY['new', 'sale'];

-- GIN index for integer arrays (user roles, permissions)
```

```sql
CREATE INDEX idx_users_roles ON users USING GIN (role_ids);
SELECT * FROM users WHERE role_ids @> ARRAY[3];  -- users with role 3
```

B-tree index on array expressions

```sql
-- Index first element (if frequently queried)
CREATE INDEX idx_products_first_tag ON products ((tags[1]));

-- Functional index for specific containment
-- (GIN is usually better for this)
```

Arrays vs JSONB vs Normalized Tables

```
Use case comparison:
──────────────────────────────────────────────────────────────────────
                    Array[]     JSONB       Normalized table
──────────────────────────────────────────────────────────────────────
Type safety         ✓ typed     ✗ mixed      ✓ typed
Element structure   flat        nested       full schema
Query complex data  limited     jsonpath     full SQL
Join capability     limited     limited      ✓ full joins
Containment search  GIN index   GIN index    B-tree index
Sorted access       array_agg   jsonb_agg    ORDER BY
Count/aggregate     unnest+agg  expand+agg   GROUP BY
Schema enforcement  weak        none         ✓ constraints
Max appropriate N   ~100        ~1000        unlimited
Foreign keys        ✗           ✗            ✓
──────────────────────────────────────────────────────────────────────

Choose Array when:
  - Small, homogeneous list of simple values
  - Order matters
  - No need for FK relationships
  - Examples: tags, permission flags, phone numbers, search keywords

Choose JSONB when:
  - Heterogeneous or nested data
  - Schema varies by row
  - Examples: product attributes, event payloads, config

Choose normalized table when:
  - Elements need their own attributes
  - Need FK integrity
  - Need aggregation, grouping, or joins on elements
  - Examples: order items, user roles, product categories
```

SQL Examples

Tags system using arrays

```
CREATE TABLE articles (
    article_id  BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    title       TEXT    NOT NULL,
    body        TEXT    NOT NULL,
    tags        TEXT[]  NOT NULL DEFAULT '{}',
    author_id   INTEGER NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_articles_tags ON articles USING GIN (tags);

-- Insert with tags
INSERT INTO articles (title, body, tags, author_id) VALUES
('PostgreSQL Arrays', 'Arrays are powerful...',
ARRAY['postgresql','database','arrays'], 1),
('JSON in PG', 'JSONB is great...', ARRAY['postgresql','json','nosql'], 1);

-- Find articles tagged 'postgresql'
SELECT title FROM articles WHERE tags @> ARRAY['postgresql'];

-- Find articles tagged either 'arrays' or 'json'
SELECT title FROM articles WHERE tags && ARRAY['arrays', 'json'];

-- Find articles tagged both 'postgresql' AND 'json'
SELECT title FROM articles WHERE tags @> ARRAY['postgresql', 'json'];

-- Count articles per tag
SELECT tag, count(*) AS article_count
FROM articles, unnest(tags) AS tag
GROUP BY 1
ORDER BY 2 DESC;

-- Add a tag
UPDATE articles SET tags = array_append(tags, 'tutorial')
WHERE article_id = 1 AND NOT 'tutorial' = ANY(tags);

-- Remove a tag
UPDATE articles SET tags = array_remove(tags, 'nosql')
WHERE article_id = 2;

-- Find the most common tags
SELECT tag, count(*) AS usage_count
FROM (
    SELECT unnest(tags) AS tag FROM articles
) t
GROUP BY 1
ORDER BY 2 DESC
LIMIT 10;
```

Phone numbers as arrays

```
CREATE TABLE contacts (
    contact_id  BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   TEXT    NOT NULL,
    phones      TEXT[]  NOT NULL DEFAULT '{}',
    emails      TEXT[]  NOT NULL DEFAULT '{}',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Find contacts with specific phone
SELECT * FROM contacts WHERE '555-1234' = ANY(phones);

-- Find contacts with multiple emails
SELECT * FROM contacts WHERE cardinality(emails) > 1;
```

Permission bitfield as integer array

```
CREATE TABLE user_permissions (
    user_id         INTEGER PRIMARY KEY,
    permission_ids  INTEGER[] NOT NULL DEFAULT '{}',
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_user_permissions ON user_permissions USING GIN (permission_ids);

-- Grant permission
UPDATE user_permissions
SET permission_ids = array_append(permission_ids, 5),
    updated_at = now()
WHERE user_id = 1 AND NOT 5 = ANY(permission_ids);

-- Check permission
SELECT EXISTS (
    SELECT 1 FROM user_permissions
    WHERE user_id = 1 AND permission_ids @> ARRAY[5]
) AS has_permission;

-- Users with all required permissions
SELECT user_id FROM user_permissions
WHERE permission_ids @> ARRAY[1, 3, 5];  -- must have all three
```

Common Mistakes

1. **Treating arrays as 0-indexed**

```
SELECT ARRAY['a','b','c'][0];  -- NULL (not 'a'!)
SELECT ARRAY['a','b','c'][1];  -- 'a'
```

2. **Not handling out-of-bounds access**

```
SELECT ARRAY[1,2,3][10];  -- NULL, not an error
-- Always check array_length() before accessing specific positions
```

3. **Using `=` for element containment**

```
-- BAD: checks if entire array equals ARRAY['tag']
WHERE tags = ARRAY['tag']

-- GOOD: checks if 'tag' is an element
WHERE 'tag' = ANY(tags)
-- or
WHERE tags @> ARRAY['tag']
```

4. **Creating arrays of arrays when you mean a flat array**

```
SELECT ARRAY[ARRAY[1,2], ARRAY[3,4]];  -- 2D array
SELECT ARRAY[1,2] || ARRAY[3,4];        -- {1,2,3,4} flat array
```

5. **Using arrays for data that should be normalized** — Arrays of foreign keys (`user_ids INTEGER[]`) cannot have FK constraints enforced. Use a junction table instead.
6. **Not indexing arrays that are frequently searched** — Queries like `WHERE 'tag' = ANY(tags)` without a GIN index are O(n) full scans.

Best Practices

1. **Use `GIN` indexes** on any array column used in containment or element-search queries.
2. **Prefer normalized tables** when array elements need FK integrity, their own attributes, or count > ~50.
3. **Use `array_agg(... ORDER BY ...)`** to build arrays with meaningful ordering from query results.
4. **Use `unnest()`** to "join" array elements back to relational queries.
5. **Add NOT NULL + DEFAULT '{}'** on array columns to avoid NULL vs empty array confusion.
6. **Document array element semantics** — arrays have no schema enforcement.
7. **Use `= ANY(array)` over `IN (...)` for dynamic arrays** — syntax is more idiomatic in PostgreSQL.

Performance Considerations

GIN index maintenance

```
-- GIN indexes are slower to update than B-tree
-- For high-write tables, tune autovacuum and consider:
ALTER TABLE articles SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_analyze_scale_factor = 0.005
);
```

```
-- Pending list: GIN buffers inserts in a fast list before merging
-- Default: gin_pending_list_limit = 4MB
-- For bulk loads, increase temporarily:
SET gin_pending_list_limit = 128000;  -- 128MB
```

Array size impact

```
-- Small arrays (< 8 bytes total): stored inline, very fast
-- Medium arrays (8 bytes - 2KB): stored inline, good performance
-- Large arrays (> 2KB): TOAST-ed, slower to access

-- Check average array sizes in your table
SELECT
    avg(cardinality(tags)) AS avg_tags,
    max(cardinality(tags)) AS max_tags,
    pg_size_pretty(avg(octet_length(tags::TEXT))::BIGINT) AS avg_storage
FROM articles;
```

Interview Questions & Answers

Q1: Are PostgreSQL arrays 0-indexed or 1-indexed?

A: 1-indexed by default. `array[1]` is the first element. You can technically create non-standard lower bounds using `array[lower_bound:upper_bound]` syntax, but this is unusual. Out-of-bounds access returns NULL rather than an error.

Q2: What is the difference between `@>`, `<@`, and `&&` for arrays?

A: `@>` means "left contains right" (left array contains all elements of right array). `<@` means "left is contained in right". `&&` means "overlap" — the arrays share at least one common element. All three can use a GIN index.

Q3: How do you efficiently search for an element in an array column?

A: Use `element = ANY(array_column)` or `array_column @> ARRAY[element]`. Both have equivalent performance with a GIN index. Without a GIN index, both require a full table scan.

Q4: When would you use an array instead of a junction table?

A: Arrays are appropriate for small, simple, homogeneous collections where you don't need FK constraints, the elements don't have their own attributes, and the collection is read and written together. Examples: tags, phone numbers, search keywords. Junction tables are appropriate when elements need FK integrity, have their own attributes, or when you need to aggregate/group on element values.

Q5: What does `unnest()` do and when would you use it?

A: `unnest()` expands an array into a set of rows. Use it when you need to apply relational operations (WHERE, GROUP BY, JOIN) to array elements. For example: `SELECT tag, count(*) FROM articles, unnest(tags) AS tag GROUP BY 1` counts articles per tag.

Q6: Can you index an array column for range queries (e.g., find all arrays where element > 5)?

A: Not directly with a GIN index — GIN supports containment and equality. For range queries on array elements, you need to either `unnest()` into a CTE or use `jsonb_path_query` if the data is JSONB. Or use a normalized table design.

Q7: What happens when you access an out-of-bounds array index in PostgreSQL?

A: PostgreSQL returns NULL without raising an error. This is by design. Always use `array_length()` or `cardinality()` to validate bounds if you need error behavior.

Q8: How do you remove duplicates from an array?

A: Use `ARRAY(SELECT DISTINCT unnest(your_array) ORDER BY 1)`. There is no built-in `array_uniq()` function in standard PostgreSQL, though some extensions provide it.

Exercises with Solutions

Exercise 1

Design a `recipes` table where each recipe has an array of ingredient names and an array of allergens. Write queries to find all recipes that contain a specific ingredient and all recipes safe for someone with a nut allergy.

Solution:

```
CREATE TABLE recipes (
    recipe_id   BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT    NOT NULL,
    ingredients TEXT[]  NOT NULL DEFAULT '{}',
    allergens   TEXT[]  NOT NULL DEFAULT '{}',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_recipes_ingredients ON recipes USING GIN (ingredients);
CREATE INDEX idx_recipes_allergens   ON recipes USING GIN (allergens);

-- Find recipes with garlic
SELECT name FROM recipes WHERE 'garlic' = ANY(ingredients);

-- Find nut-free recipes (no 'nuts', 'peanuts', 'tree nuts')
SELECT name FROM recipes
WHERE NOT (allergens && ARRAY['nuts', 'peanuts', 'tree nuts', 'almonds',
'cashews']);

-- Count recipes per allergen
SELECT allergen, count(*) AS recipe_count
FROM recipes, unnest(allergens) AS allergen
GROUP BY 1 ORDER BY 2 DESC;
```

Exercise 2

Write a function that merges two arrays, removes duplicates, and returns the sorted result.

Solution:

```sql
CREATE OR REPLACE FUNCTION array_merge_unique(arr1 ANYARRAY, arr2 ANYARRAY)
RETURNS ANYARRAY
LANGUAGE SQL IMMUTABLE AS $$
    SELECT ARRAY(
        SELECT DISTINCT unnest(arr1 || arr2)
        ORDER BY 1
    );
$$;

SELECT array_merge_unique(ARRAY[3,1,2,1], ARRAY[2,4,3,5]);
-- {1,2,3,4,5}
```

Cross-References

- `05_json_jsonb.md` — JSON arrays vs PostgreSQL native arrays
- `07_special_types.md` — special array-like types (tsvector)
- `08_constraints.md` — constraints cannot reference array elements directly
- `../07_Indexes/` — GIN index deep dive
- `../06_Database_Design/04_entity_relationships.md` — when to normalize instead of using arrays